

Báo Cáo Lỗi Phần Mềm (Bug Report) — Hệ Thống EShop SUT

Người kiểm thử: PHẠM ĐỨC TOÀN

MSSV: 23127540 — Lớp: 23KTPM2

Môn học: CS423 / CSC13003 – Kiểm chứng Phần mềm (HW06-AI)

Kho mã nguồn SUT: [eshop-sut](#)

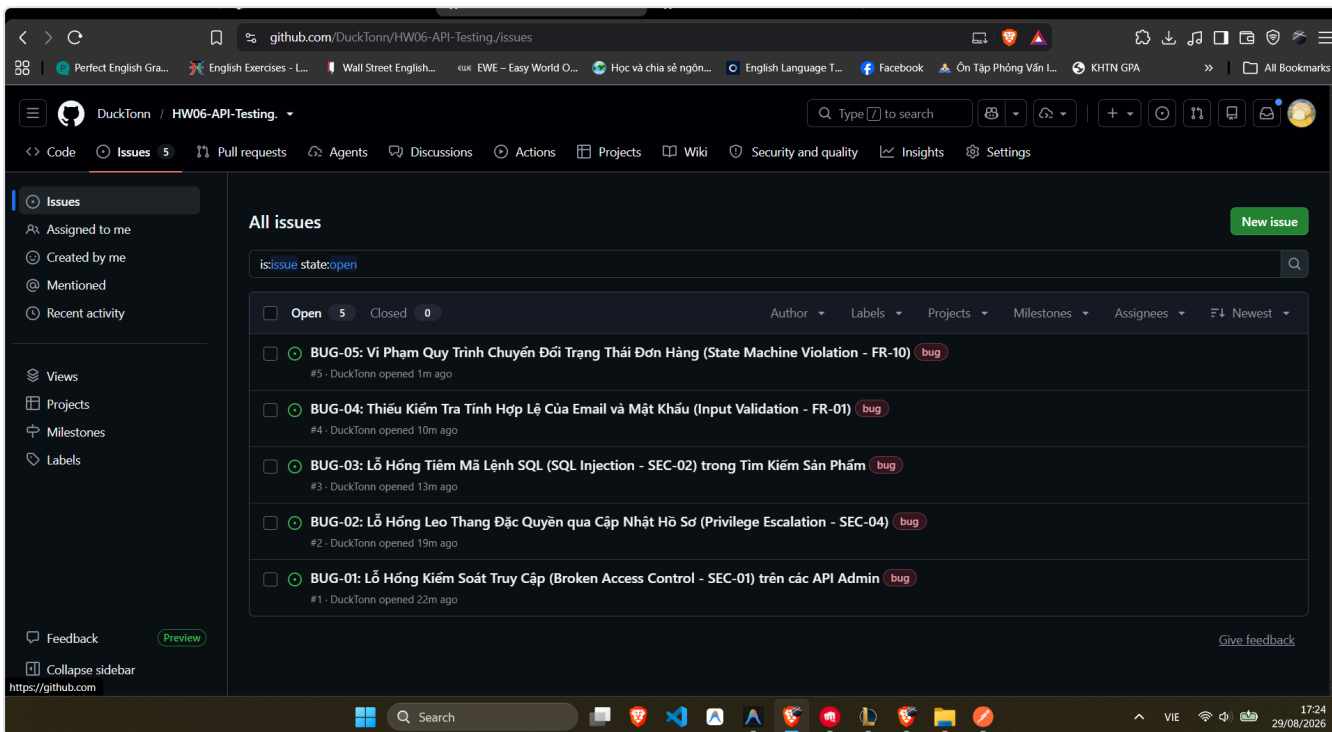
Ngày lập báo cáo: 19/08/2026

1. Tóm Tắt Tổng Quan (Executive Summary)

Trong quá trình thực thi kiểm thử tự động toàn diện qua các phân vùng **Pool A** (FR-01: Đăng ký tài khoản, FR-06: Chi tiết sản phẩm), **Pool B** (FR-07: Giỏ hàng), và **Pool C** (FR-12: Phân quyền Web Admin), chúng tôi đã phát hiện **5 lỗi nghiêm trọng và lỗ hổng bảo mật** trong mã nguồn backend `eshop-sut/backend/server.js`.

Tất cả các lỗi đã được lập trình kịch bản kiểm thử trong Postman Collection và tự động phát hiện (FAIL) trong quy trình CI/CD Newman. Toàn bộ 5 lỗi đã được mở issues và gắn nhãn tương ứng trên kho lưu trữ GitHub:

- Danh sách GitHub Issues chính thức: <https://github.com/DuckTonn/HW06-API-Testing/issues>

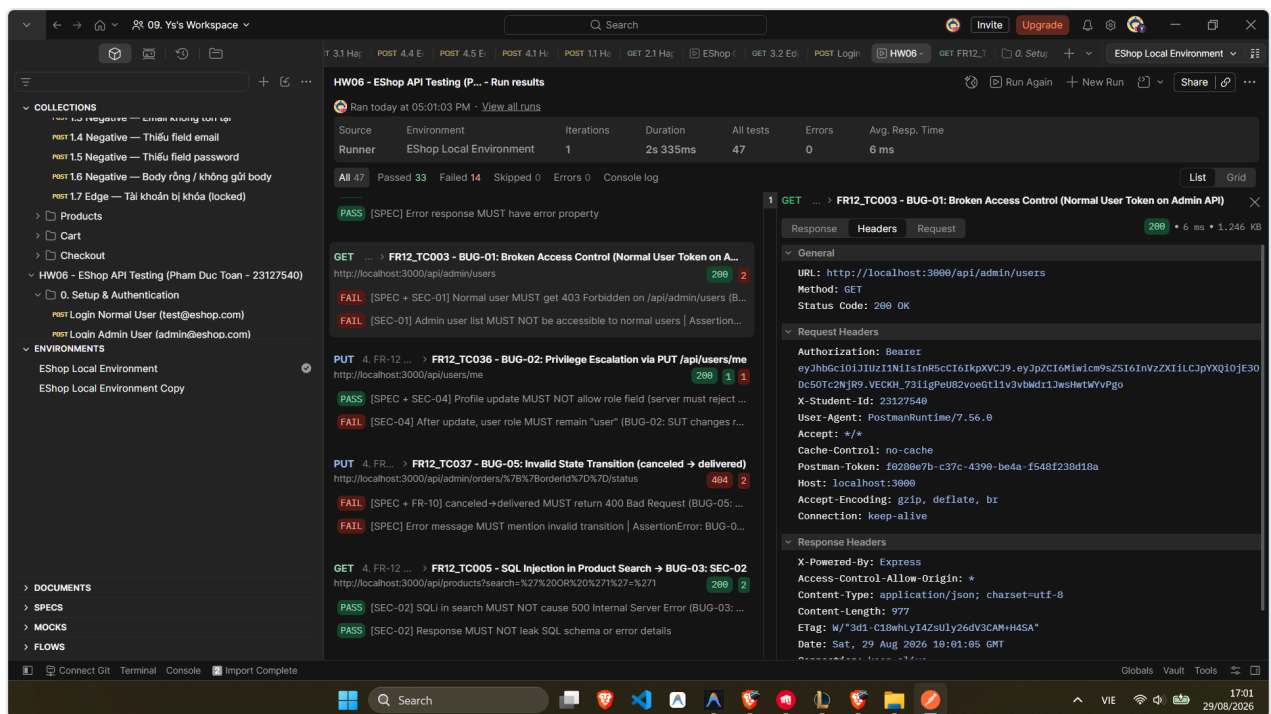


2. Chi Tiết Danh Mục Lỗi Phát Hiện

🐛 BUG-01: Lỗ Hổng Kiểm Soát Truy Cập (Broken Access Control - SEC-01) trên các API Admin

- Mức độ nghiêm trọng:** NGHIÊM TRỌNG (CRITICAL)
- Phân loại:** An ninh bảo mật / Kiểm soát phân quyền (Authorization)
- Các Endpoint bị ảnh hưởng:**
 - GET `/api/admin/users` (Lấy danh sách toàn bộ người dùng)
 - DELETE `/api/admin/users/:id` (Xóa tài khoản người dùng bất kỳ)
 - GET `/api/admin/orders` (Xem danh sách đơn hàng toàn hệ thống)
 - PUT `/api/admin/orders/:id/status` (Cập nhật trạng thái đơn hàng)
 - POST `/api/admin/coupons` (Tạo mã giảm giá mới)
 - DELETE `/api/admin/coupons/:id` (Xóa mã giảm giá)

- **Mô tả lỗi:** Middleware `authenticateToken` chỉ kiểm tra tính hợp lệ của chữ ký JWT token nhưng **hoàn toàn không kiểm tra vai trò người dùng** (`req.user.role === 'admin'`). Do đó, bất kỳ tài khoản người dùng thông thường (`role: "user"`) nào sau khi đăng nhập đều có thể gửi request đến tất cả các API quản trị của Admin, đọc/xóa dữ liệu người dùng khác và thay đổi trạng thái đơn hàng.
- **Các bước tái hiện (Steps to Reproduce):**
- Đăng ký tài khoản người dùng thông thường qua `POST /api/register`.
- Đăng nhập qua `POST /api/login` và nhận chuỗi JWT `userToken`.
- Gửi request `GET /api/admin/users` kèm Header `Authorization: Bearer <userToken>`.
- **Kết quả thực tế (Observed Result):** Server trả về mã HTTP `200 OK` kèm theo toàn bộ danh sách người dùng, email và vai trò trong hệ thống.
- **Kết quả kỳ vọng (Expected Result):** Server bắt buộc phải từ chối truy cập và trả về mã lỗi HTTP `403 Forbidden` do tài khoản không có quyền Admin.
- **Đề xuất khắc phục:** Thêm middleware kiểm tra quyền `requireAdmin`: `javascript function requireAdmin(req, res, next) { if (req.user && req.user.role === 'admin') { next(); } else { res.status(403).json({ error: 'Access denied. Admin role required.' }); } }`
- **GitHub Issue:** [#1 — BUG-01: Broken Access Control \(SEC-01\) trên các API Admin](#)
- **Minh chứng kiểm thử & Newman Assertion:**



🐛 BUG-02: Lỗ Hổng Leo Thang Đặc Quyền qua Cập Nhật Hồ Sơ (Privilege Escalation - SEC-04)

- **Mức độ nghiêm trọng:** **NGHIÊM TRỌNG (CRITICAL)**
- **Phân loại:** Gán quyền trái phép (Mass Assignment / Role Escalation)
- **Endpoint bị ảnh hưởng:** `PUT /api/users/me`
- **Mô tả lỗi:** Backend cho phép client tùy ý truyền trường `role` trong payload JSON của API cập nhật thông tin cá nhân. Câu lệnh SQL cập nhật trực tiếp trường `users.role` trong cơ sở dữ liệu SQLite mà không có danh sách trắng (whitelist) các trường được phép sửa.
- **Các bước tái hiện (Steps to Reproduce):**
- Đăng nhập với tài khoản người dùng thông thường (`role = "user"`).
- Gửi request `PUT /api/users/me` kèm payload JSON: `json { "name": "Attacker", "role": "admin" }`
- Gọi lại API `GET /api/users/me` để kiểm tra thông tin tài khoản.
- **Kết quả thực tế (Observed Result):** Server trả về `200 OK` thông báo "Profile updated" và cập nhật trường `role` của người dùng thành `admin` trong cơ sở dữ liệu.

- **Kết quả kỳ vọng (Expected Result):** Server phải loại bỏ (strip) hoặc từ chối trường `role` trong payload cập nhật hồ sơ cá nhân và trả về mã lỗi `400 Bad Request`.
- **Đề xuất khắc phục:** Chỉ cho phép cập nhật các trường thông tin cơ bản (`name` , `shipping_address` , `phone`), tuyệt đối không nhận trường `role` từ client.
- **GitHub Issue:** [#2 — BUG-02: Privilege Escalation qua Cập Nhật Hồ Sơ \(SEC-04\)](#)
- **Minh chứng kiểm thử & Postman Payload:**

The screenshot displays a REST client interface with the following details:

- Test Suite:** HW06 - EShop API Testing (P... - Run results)
- Environment:** EShop Local Environment
- Summary:** All 47 tests passed, 33 passed, 14 failed, 0 skipped, 0 errors, 6 ms avg. resp. time.
- Test Results:**
 - GET ... > FR12_TC003 - BUG-01: Broken Access Control (Normal User Token on A...**
 - FAIL [SPEC + SEC-01] Normal user MUST get 403 Forbidden on /api/admin/users (B...
 - FAIL [SEC-01] Admin user list MUST NOT be accessible to normal users | Assertion...
 - PUT 4. FR-12 ... > FR12_TC036 - BUG-02: Privilege Escalation via PUT /api/users/me**
 - PASS [SPEC + SEC-04] Profile update MUST NOT allow role field (server must reject ...
 - FAIL [SEC-04] After update, user role MUST remain "user" (BUG-02: SUT changes r...
 - PUT 4. FR... > FR12_TC037 - BUG-05: Invalid State Transition (canceled → delivered)**
 - FAIL [SPEC + FR-10] canceled→delivered MUST return 400 Bad Request (BUG-05: ...
 - FAIL [SPEC] Error message MUST mention invalid transition | AssertionError: BUG-0...
 - GET 4. FR-12 ... > FR12_TC005 - SQL Injection in Product Search → BUG-03: SEC-02**
 - PASS [SEC-02] SQLi in search MUST NOT cause 500 Internal Server Error (BUG-03: ...
 - PASS [SEC-02] Response MUST NOT leak SQL schema or error details
- Request Details (PUT /api/users/me):**

```

1 {
2   "name": "Escalated Admin User",
3   "shipping_address": "227 Nguyen Van Cu, Q5, TP.HCM",
4   "phone": "0912345678",
5   "role": "admin"
6 }
```

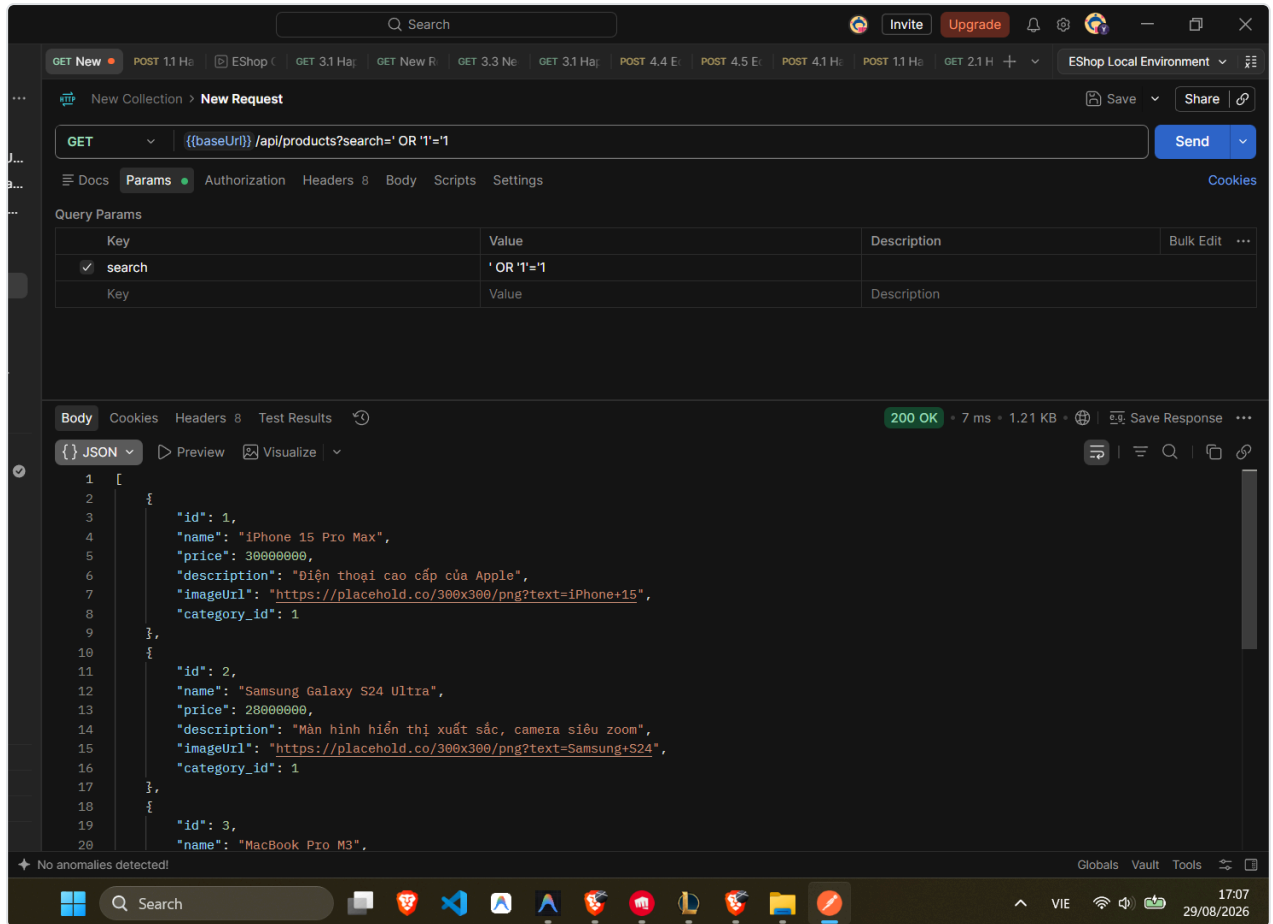
🦋 BUG-03: Lỗ Hổng Tiêm Mã Lệnh SQL (SQL Injection - SEC-02) trong Tìm Kiếm Sản Phẩm

- **Mức độ nghiêm trọng:** CAO (HIGH)
- **Phân loại:** An ninh bảo mật / SQL Injection
- **Endpoint bị ảnh hưởng:** `GET /api/products?search=...`
- **Mô tả lỗi:** Backend sử dụng phép nối chuỗi trực tiếp (template literal) để tạo câu truy vấn SQL: `const query = 'SELECT * FROM products WHERE name LIKE \'%' + searchQuery + '%\''`; thay vì sử dụng cơ chế tham số hóa (Parameterized Query). Lỗ hổng này cho phép kẻ tấn công chèn các biểu thức SQL tùy ý, dẫn đến 2 hành vi nguy hiểm:
- **Boolean-based SQL Injection:** Truyền payload `' OR '1'='1` khiến điều kiện SQL luôn đúng (`TRUE`), bẻ gãy bộ lọc tìm kiếm và đổ toàn bộ danh mục sản phẩm (dump full database) về client với mã `200 OK`.
- **Error-based SQL Injection / Information Disclosure:** Truyền ký tự bẻ gãy cú pháp như `'` khiến câu truy vấn lỗi cú pháp, máy chủ sập và trả về mã lỗi `500 Internal Server Error` kèm trang HTML rò rỉ cấu trúc SQLite nội bộ (`<h1>Database Error</h1><p>SQLITE_ERROR: unrecognized token: "'</p>`).
- **Các bước tái hiện (Steps to Reproduce):**
- **Trường hợp A (Rò rỉ lỗi Database):** Gửi request `GET /api/products?search='`
- **Trường hợp B (Dump toàn bộ Database):** Gửi request `GET /api/products?search=' OR '1'='1`
- **Kết quả thực tế (Observed Result):**
- Trường hợp A: Trả về `500 Internal Server Error` kèm giao diện HTML để lộ thông tin lỗi SQLite backend.
- Trường hợp B: Trả về `200 OK` nhưng trả về toàn bộ danh sách tất cả sản phẩm trong database thay vì kết quả tìm kiếm rỗng.
- **Kết quả kỳ vọng (Expected Result):** Hệ thống bắt buộc phải dùng Parameterized Query để xử lý an toàn đầu vào:


```
javascript const query = "SELECT * FROM products WHERE name LIKE ?"; db.all(query, [`%${searchQuery}%`],
```

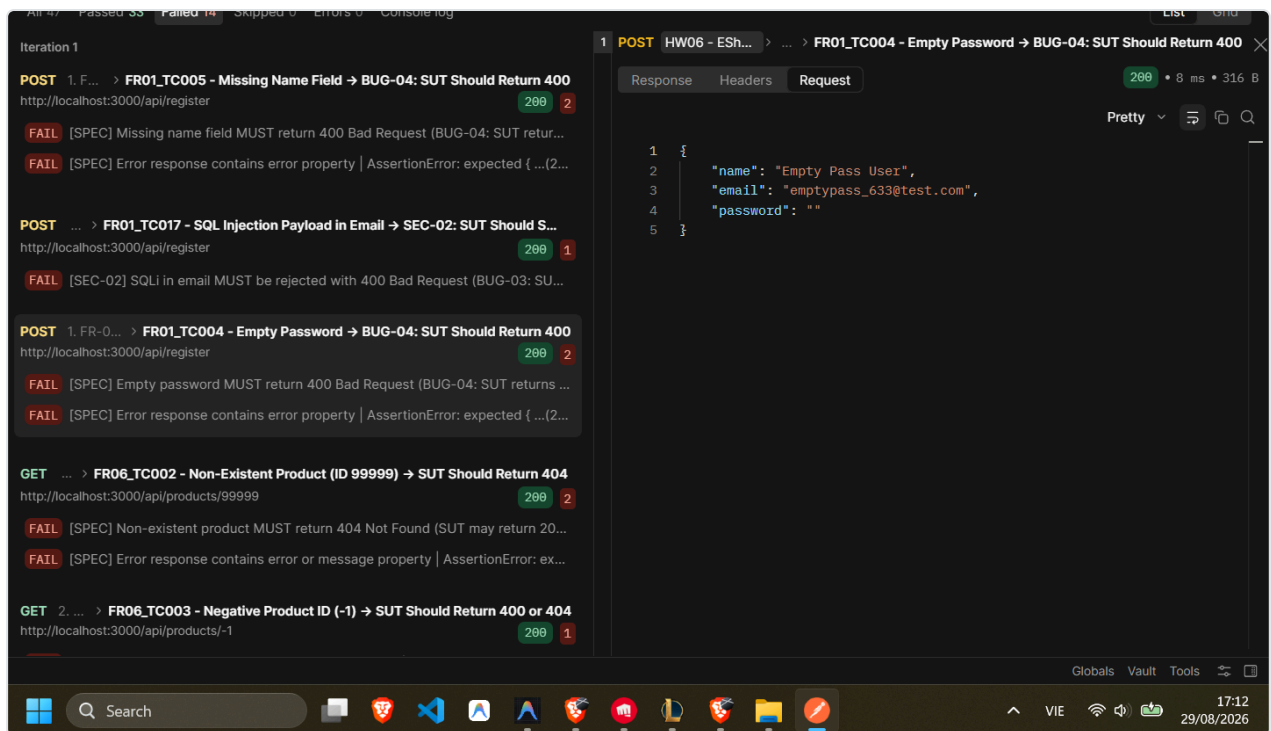
(err, rows) => { ... }); Khi tìm kiếm chuỗi không khớp hoặc chứa ký tự đặc biệt, server phải trả về danh sách rỗng [] với mã 200 OK hoặc 400 Bad Request, tuyệt đối không thực thi cú pháp SQL ngoại lai hay trả về lỗi 500.

- **GitHub Issue:** [#3 — BUG-03: SQL Injection trong Tìm Kiếm Sản Phẩm \(SEC-02\)](#)
- **Minh chứng kiểm thử & SQL Dump:**



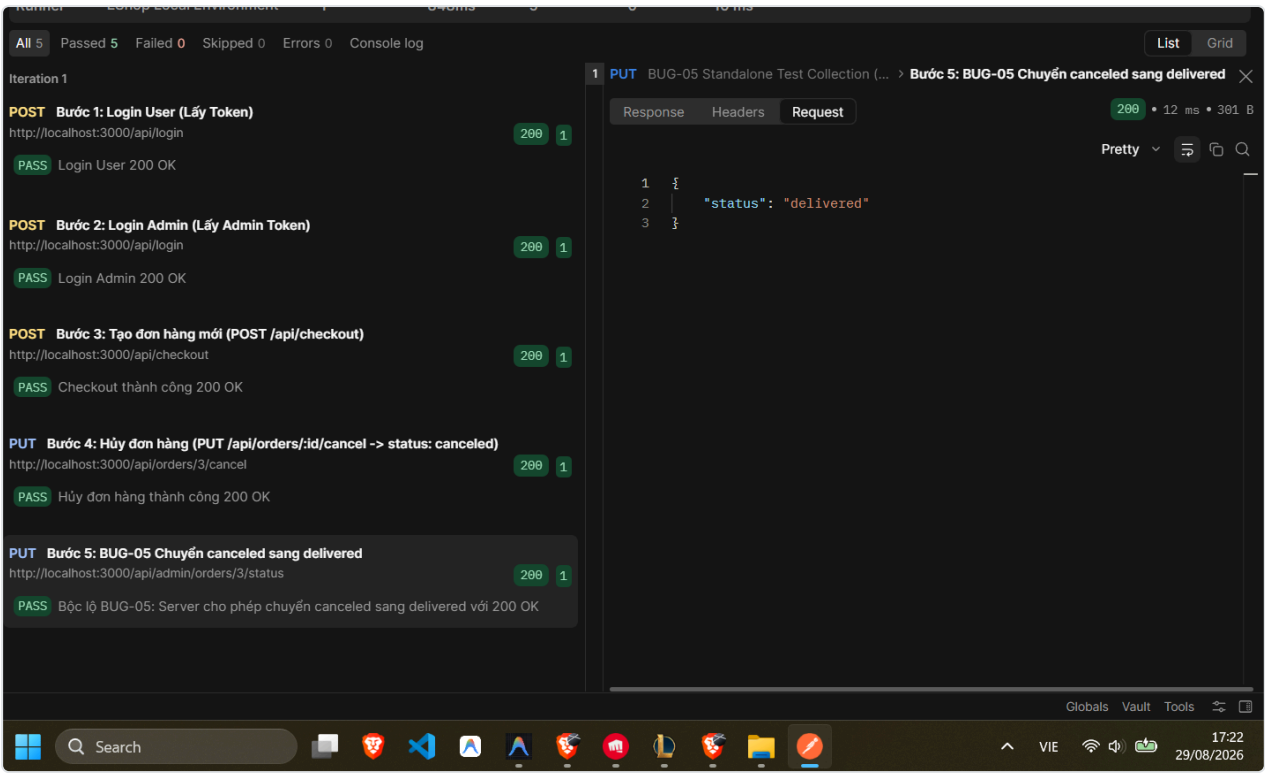
🐛 BUG-04: Thiếu Kiểm Tra Tính Hợp Lệ Của Email và Mật Khẩu (Input Validation - FR-01)

- **Mức độ nghiêm trọng:** TRUNG BÌNH (MEDIUM)
- **Phân loại:** Kiểm tra dữ liệu đầu vào (Input Validation / Boundary Value)
- **Endpoint bị ảnh hưởng:** POST /api/register
- **Mô tả lỗi:** API đăng ký tài khoản không kiểm tra định dạng email theo chuẩn RFC 5322, không kiểm tra độ dài tối thiểu hay độ phức tạp của mật khẩu.
- **Các bước tái hiện (Steps to Reproduce):**
 - Gửi request POST /api/register với payload: `json { "name": "Test User", "email": "invalid_email_format", "password": "1" }`
 - **Kết quả thực tế (Observed Result):** Server trả về 200 OK với thông báo "User registered successfully" và lưu tài khoản không hợp lệ vào hệ thống.
 - **Kết quả kỳ vọng (Expected Result):** Server phải từ chối và trả về mã lỗi 400 Bad Request kèm thông điệp lỗi rõ ràng.
- **GitHub Issue:** [#4 — BUG-04: Thiếu Kiểm Tra Tính Hợp Lệ Của Email và Mật Khẩu \(FR-01\)](#)
- **Minh chứng kiểm thử:**



🐛 BUG-05: Vi Phạm Quy Trình Chuyển Đổi Trạng Thái Đơn Hàng (State Machine Violation - FR-10)

- **Mức độ nghiêm trọng: CAO (HIGH)**
- **Phân loại:** Lỗi logic nghiệp vụ / State Transition
- **Endpoint bị ảnh hưởng:** `PUT /api/admin/orders/:id/status`
- **Mô tả lỗi:** Mã nguồn backend có logic cho phép đơn hàng đã bị hủy (`canceled`) được chuyển sang trạng thái đã giao hàng (`delivered`). Điều này vi phạm nghiêm trọng máy trạng thái đơn hàng (đơn hàng đã hủy là trạng thái kết thúc - terminal state, không thể tiếp tục giao).
- **Các bước tái hiện (Steps to Reproduce):**
 - Lấy một đơn hàng đang ở trạng thái `canceled` .
 - Gửi request `PUT /api/admin/orders/:id/status` với body `{"status": "delivered"}` .
 - **Kết quả thực tế (Observed Result):** Server trả về `200 OK` , cập nhật trạng thái đơn hàng thành `delivered` .
 - **Kết quả kỳ vọng (Expected Result):** Server phải chặn giao dịch và trả về mã lỗi `400 Bad Request` với thông điệp `"Invalid state transition from canceled to delivered"` .
- **GitHub Issue:** [#5 — BUG-05: Vi Phạm Quy Trình Chuyển Đổi Trạng Thái Đơn Hàng \(FR-10\)](#)
- **Minh chứng kiểm thử:**



3. Tổng Hợp Mức Độ Nghiêm Trọng

Mã Lỗi	Tên Lỗi	Endpoint	Mức Độ	Trạng Thái Phát Hiện
BUG-01	Broken Access Control trên API Admin	/api/admin/*	CRITICAL	Đã phát hiện qua Test Suite
BUG-02	Privilege Escalation qua cập nhật hồ sơ	PUT /api/users/me	CRITICAL	Đã phát hiện qua Test Suite
BUG-03	SQL Injection trong tìm kiếm sản phẩm	GET /api/products	HIGH	Đã phát hiện qua Test Suite
BUG-04	Thiếu validation email và mật khẩu	POST /api/register	MEDIUM	Đã phát hiện qua Test Suite
BUG-05	Cho phép chuyển trạng thái canceled -> delivered	PUT /api/admin/orders/:id/status	HIGH	Đã phát hiện qua Test Suite